

Understanding the loss function

From two motion predictions to one number that guides learning.

Prepared for Sarel Ye · 16 September 2026

The concrete idea: a loss function compares a prediction with a reference answer and returns a number. That number expresses how strongly this training objective penalizes the mistake. Training tries to reduce it by adjusting the network's parameters.

Start where your neural network guide ended

You have followed a complete forward pass: inputs become hidden values, then two outputs. The next question is: "How wrong are those outputs, and how should the two mistakes count together?"

(velocity, force) → network → (displacement, final velocity)
predictions + reference labels → loss

How this supplement connects to Lesson 8

Lesson 8, "Design the loss around the task," introduces scaled squared errors. We will unpack every part of that formula, reuse the numerical prediction from your neural network supplement, and calculate a batch loss on paper.

We follow your latest task: predict signed displacement $\mathbf{d}$ during a fixed interval and velocity $\mathbf{v}$ at its end. Mass, friction, gravity, and the interval stay fixed. This is a mathematical walkthrough; the predictions are illustrative, and no model has been trained here.

Your route through the explanation

Signed error → squaring → physical scales → one complete calculation → batch averaging → priorities → alternatives → training feedback → notebook checks.

What exactly are we measuring?

A prediction is the network's answer. A label is the reference answer for the same input and the same time interval.

Use the experiment already worked in both guides: start at rest, apply 10 N for 0.1 s, with mass 2 kg, static friction coefficient 0.4, kinetic friction coefficient 0.2, and gravity 9.8 m/s². The reference acceleration is 3.04 m/s², giving the labels below.

Quantity	Toy prediction	Reference label	Prediction – label
Displacement d	0.013 m	0.0152 m	–0.0022 m
Final velocity v	0.260 m/s	0.304 m/s	–0.044 m/s

These are exactly the outputs from page 5 of your neural network supplement. There, the first output was next position. Because that example starts at $x = 0$, its numerical next position also equals displacement.

Why not simply add signed errors?

Imagine two displacement errors: +0.01 m and –0.01 m. Their mean is zero, even though both predictions are wrong. A signed mean measures bias, or a tendency to overpredict or underpredict; it does not measure overall error size.

$$\begin{aligned} (+0.01 + (-0.01)) / 2 &= 0 \\ (0.01^2 + (-0.01)^2) / 2 &= 0.0001 \text{ m}^2 \end{aligned}$$

Squaring makes both contributions nonnegative. Doubling an error multiplies its squared contribution by four. Large mistakes therefore matter more strongly than small ones.

Keep three things separate: the signed error tells you direction; its magnitude tells you size; the loss tells you how your chosen objective penalizes the errors.

Vocabulary: label = 标签; prediction error = 预测误差; loss function = 损失函数.

Why divide by a scale?

Displacement is measured in meters. Velocity is measured in meters per second. Their raw squared errors do not have the same units.

Adding a number in m^2 to a number in m^2/s^2 has no natural physical meaning. Even as a numerical score, it would depend arbitrarily on your unit convention: changing meters to centimeters multiplies the displacement's numerical squared error by 10,000.

Make each error dimensionless

$$e_d = (\hat{d} - d) / s_d$$

$$e_v = (\hat{v} - v) / s_v$$

Symbol	Concrete meaning
$\hat{d}, \hat{v}$	Predicted displacement and final velocity. The hat means "predicted."
d, v	Reference labels at the end of the specified interval; v here is not initial velocity.
s_d, s_v	Fixed positive error scales, with units m and m/s respectively.
e_d, e_v	Scaled errors: signed, dimensionless numbers.

Use the guide's illustrative scales: $s_d = 0.01 \text{ m}$ and $s_v = 0.1 \text{ m/s}$. An error of 0.005 m becomes 0.5. A velocity error of 0.02 m/s becomes 0.2. These numbers say how many chosen scales fit into each error.

A scale is a choice, not a learned network weight

A smaller scale penalizes the same physical error more strongly. Halving s_d doubles e_d and quadruples its squared contribution. Select scales from meaningful physical sizes or training-data statistics, then keep them fixed for the run. Fit any data-derived statistics on training data only.

The illustrative scales are **not established acceptance tolerances** for your project. A total loss below 1 does not guarantee that every prediction is acceptable.

Follow the same cuboid prediction

Keep the toy predictions from page 2. Choose $s_d = 0.01$ m, $s_v = 0.1$ m/s, and equal weighting for the two scaled errors.

First subtract, then scale, then square

Operation	Displacement	Final velocity
Subtract label	$0.013 - 0.0152$ $= -0.0022$ m	$0.260 - 0.304$ $= -0.044$ m/s
Divide by scale	$-0.0022 / 0.01$ $= -0.22$	$-0.044 / 0.1$ $= -0.44$
Square	$(-0.22)^2 = 0.0484$	$(-0.44)^2 = 0.1936$

$$\begin{aligned}\ell &= e_d^2 + e_v^2 \\ \ell &= 0.0484 + 0.1936 = 0.2420\end{aligned}$$

The symbol ℓ , read "ell," denotes the loss for this one example. It is dimensionless. The velocity term contributes four times as much as the displacement term because its scaled error has twice the magnitude.

What does 0.2420 tell you?

It is a score under this particular definition. It is not 24.2% error, a probability of failure, or a distance. To understand the physical mistake, still report the two errors: -0.0022 m and -0.044 m/s.

If both physical errors were halved while the scales stayed fixed, each squared term would become one quarter as large. The loss would fall to 0.0605. If both outputs exactly matched their labels, the loss would be zero.

One loss value does not uniquely describe the prediction. Opposite error signs produce the same squared loss. Different combinations of displacement and velocity errors can also produce the same total.

Connect back to the original lesson

Lesson 8 uses errors of 0.005 m and 0.02 m/s: $(0.005 / 0.01)^2 + (0.02 / 0.1)^2 = 0.25 + 0.04 = 0.29$. It is the same calculation with different example errors.

Read the full formula

$$\ell_i = e_{d,i}^2 + \lambda e_{v,i}^2$$

$$L = (1/N) \sum_{i=1}^N \ell_i$$

The index i labels an example. N is the number of examples being averaged. The symbol $\sum$ means “add the following values.” The Greek letter λ , pronounced “lambda,” is a nonnegative weight on velocity error. L is the average loss; ℓ_i is one example’s loss.

Calculate a batch of three

With the same scales and $\lambda = 1$, suppose the scaled errors are:

Example	e_d	e_v	$\ell = e_d^2 + e_v^2$
A: previous example	-0.22	-0.44	0.242
B: illustrative	+0.50	-0.20	0.290
C: exact prediction	0	0	0

$$L = (0.242 + 0.290 + 0) / 3 \approx 0.17733$$

A mean makes losses comparable across different batch sizes drawn from the same distribution: repeating every example twice leaves the mean unchanged. A sum would double. The composition of a batch still matters; easier examples can lower its mean.

A small convention that can cause confusion

This guide sums the two output terms, then averages over examples, matching Lesson 8. An MSE operation that averages over all $2N$ scalar output errors instead gives 0.08867 here, exactly half as much when $\lambda = 1$.

That factor does not change which predictions minimize the objective, but it changes gradient magnitudes and reported values. Always state the averaging convention before comparing loss curves.

For a whole dataset, weight batch means by their example counts. A small final batch should not count as much as a full batch. Equivalently, add every example’s loss and divide by the total number of examples.

Scales and weights express priorities

Equal weighting means $\lambda = 1$ for the **scaled** squared errors. It does not mean one meter of error and one meter per second of error count equally.

$$\ell = (d^* - d)^2 / s_d^2 + \lambda (\hat{v} - v)^2 / s_v^2$$

This form exposes the effective weights: $1/s_d^2$ for displacement and λ/s_v^2 for velocity. Choosing a scale already changes the priorities, even before you change λ .

Two predictions with the same total loss

Suppose the reference says the cuboid is at rest for the interval: $d = 0$ and $v = 0$. Keep $s_d = 0.01$ m and $s_v = 0.1$ m/s.

Toy result	Displacement error	Velocity error	$\lambda = 1$	$\lambda = 4$
A	+0.01 m	0 m/s	1	1
B	0 m	+0.1 m/s	1	4

A invents displacement but ends at zero speed. B ends at the correct position but invents motion. Both violate the reference resting behavior. With $\lambda = 1$ their total losses are equal; with $\lambda = 4$ the objective penalizes B more strongly.

Choose once, inspect, then revise with evidence

Start with the scaled squared-error objective and $\lambda = 1$. Inspect displacement and velocity errors separately on validation experiments. If stopping failures matter for the intended task, use that evidence to reconsider scales, weighting, or which cases appear in training.

Setting $\lambda = 0$ removes the direct penalty on velocity error. Increasing it can favor velocity accuracy at the expense of displacement when the model cannot fit both perfectly. It does not guarantee a particular physical outcome.

Do not learn λ by simply minimizing this same loss. With λ constrained to be nonnegative, shrinking it toward zero can reduce the score without improving velocity predictions. Here, scales and λ are chosen settings, not network parameters.

Different penalties, different emphasis

For one dimensionless error e , squared error, absolute error, and Huber loss all penalize disagreement. They grow differently as the error becomes large.

Penalty	Rule	Meaning here
Squared error	e^2	Smooth; strongly emphasizes large errors. A clear first choice for checked simulator labels.
Absolute error	$ e $	Grows linearly; less dominated by large errors. Has a corner at zero.
Huber, threshold δ	$\frac{1}{2}e^2$ if $ e \leq \delta$; $\delta(e - \frac{1}{2}\delta)$ otherwise	Quadratic near zero, linear far away. δ is a chosen positive threshold in scaled-error units.

Make the growth concrete

$ e $	Squared	Absolute	Huber, $\delta = 1$
0	0	0	0
0.5	0.25	0.5	0.125
1	1	1	0.5
2	4	2	1.5
4	16	4	3.5

Look at the growth within each column, not which column has the smallest raw number. Multiplying any loss by a positive constant changes its numerical scale without changing its preferred predictions.

When would you change the first choice?

For trustworthy deterministic simulation labels, begin with squared error. With noisy measurements or occasional corrupted labels, absolute or Huber loss may help. First investigate unusual errors: a rare stopping event can be important physics, not bad data to suppress.

These are regression losses: your outputs are numerical motion quantities. You do not need a classification loss or an RL reward to train this predictor.

Vocabulary: mean squared error (MSE) = 均方误差; mean absolute error (MAE) = 平均绝对误差. The “mean” describes averaging across the chosen examples or components.

A score becomes a learning signal

The loss value measures disagreement. Its derivatives tell the optimizer how small changes to predictions and parameters would locally change that value.

Look at one scaled output

Let $p = d/s_d$ be a scaled prediction and $t = d/s_d$ its scaled label. For this illustration, the network output is expressed in this same scale. Its displacement loss is:

$$\begin{aligned}\ell_d &= (p - t)^2 \\ \partial \ell_d / \partial p &= 2(p - t)\end{aligned}$$

In our worked example, $p = 1.30$ and $t = 1.52$, so the derivative is -0.44 . Its negative sign says that a small increase in p would reduce this term. If p were too large, the derivative would be positive instead.

Follow the feedback to one weight

Suppose this scaled output contains the term wh , where $h = 0.7$ is a fixed hidden value for this forward pass, and w is an output weight. Changing w changes p . The chain rule gives:

$$\begin{aligned}\partial \ell_d / \partial w &= (\partial \ell_d / \partial p)(\partial p / \partial w) \\ &= (-0.44) \times 0.7 = -0.308\end{aligned}$$

This is a local, one-output illustration. It uses the loss scale 0.01 m, not the 1 m output scale in the earlier neural network example. For an actual batch, gradients are averaged; a shared hidden weight can receive contributions from both outputs.

Component	Its job
Loss function	Turn predictions and labels into the chosen penalty.
Backpropagation	Calculate parameter derivatives through the chain of calculations.
Optimizer	Use gradients and its update rule to adjust parameters.

You choose the objective. Backpropagation calculates gradients. The optimizer changes weights. Lesson 9 develops the update itself; a smaller batch loss does not guarantee better unseen predictions.

Low loss is evidence to inspect

Training, validation, and test losses have different jobs

Training loss guides parameter updates. Validation loss helps you choose settings and diagnose problems. Test loss evaluates the final choice on untouched examples. Split by complete trajectories or experiment conditions so neighboring transitions do not leak across sets.

Use the same scales, weights, and averaging convention when comparing candidates. If you change the objective, recompute comparable evaluation scores; a smaller number under a new definition does not by itself mean a better predictor.

Report errors in physical units too

Evidence	What it lets you inspect
Displacement / position error in m	How far the prediction misses, including signed bias.
Velocity error in m/s	Whether the cuboid is predicted to move or stop correctly.
Errors by motion regime	Rest, sliding, braking, and stopping failures hidden by a mean.
Repeated rollout at a stated horizon	How errors accumulate when predictions become the next inputs.

For each output, MAE is the mean absolute physical error. RMSE is the square root of its mean squared physical error. Both recover the output's units; report them separately rather than taking the square root of a mixed-output loss and calling it meters.

A good one-step score does not ensure a good rollout

A rollout repeatedly feeds predicted velocity into the next interval, supplies the intended force, and adds predicted displacement to position. Mistakes can accumulate and change later inputs. Inspect rest drift, stopping time, and any reversal caused by friction alone.

If one-step training gives good local predictions but poor rollouts, later consider a multi-step loss that compares an unrolled predicted trajectory with reference labels. Begin with one-step training so you can diagnose the simpler problem first.

Why displacement loss matches next-position loss:

$(x + d) - (x + d) = d - d$, provided the same starting x is used. In a rollout, the predicted starting x may already be wrong, so accumulated position error must also be checked.

The current physics.py predicts total push-and-coast displacement from rest. It does not yet supply arbitrary fixed-interval labels. A small loss against incorrect labels would still teach their mistakes.

Can you calculate and explain it?

Try these in your notebook before reading the answers. Keep $s_d = 0.01$ m, $s_v = 0.1$ m/s, and $\lambda = 1$ unless stated otherwise.

1. A prediction has displacement error $+0.01$ m and zero velocity error. What is its loss?
2. The errors are $+0.005$ m and -0.02 m/s. Calculate both contributions and their total. Then use $\lambda = 4$.
3. Keep question 2's physical errors, but halve only s_d . What is the new total with $\lambda = 1$?
4. Two examples have individual losses 0.29 and 0.71. What is their batch loss? What if every example is repeated twice?
5. A scaled prediction $p = 0.8$ has label $t = 1.0$. Calculate $(p - t)^2$ and its derivative with respect to p .
6. Why can two models with the same loss behave differently at rest? Why is a loss of zero on training data insufficient?

Answers and reasoning

1. $(0.01 / 0.01)^2 + 0 = 1$.
2. The contributions are $0.5^2 = 0.25$ and $(-0.2)^2 = 0.04$, giving **0.29**. With $\lambda = 4$, the total is $0.25 + 4 \times 0.04 = \mathbf{0.41}$.
3. The new displacement scale is 0.005 m. Its term becomes $1^2 = 1$, so the total is **1.04**. Only that term quadruples.
4. $(0.29 + 0.71) / 2 = \mathbf{0.50}$. Repeating both examples leaves the mean at 0.50.
5. The loss is **0.04**; the derivative is **-0.4**. A small increase in p locally reduces the loss.
6. A total hides the error split, signs, and distribution across cases. One model can drift in position while another invents velocity. Exact training fits do not prove accuracy on unseen inputs, reliable rollouts, or correct labels.

Your checkpoint: reproduce the 0.2420 worked loss, explain why its terms are dimensionless, and state what scales, λ , and averaging do. Then describe what you would inspect besides the total loss before trusting transmodel.

Sources and scope

Primary reference: [transmodel_learning_guide.pdf](#), Lesson 08, "Design the loss around the task," page 8; Lessons 03, 06, 07, 09, and 10 supply related context. Style and worked predictions follow [neuron_network_supplementary.pdf](#), especially page 5. The output convention follows [learning_note.md](#). Repository code was read on 16 September 2026.

The batch examples, trade-off table, and exercises are illustrative teaching material. No training or validation results are claimed. Suggested scales are not project acceptance criteria. This supplement leaves the simulation and learning plan unchanged.